

Oct 10, 2024

Orchestrating Agents: Routines and Handoffs

This recipe is archived and may reference outdated models or APIs.



Ilan Bigio (OpenAI)

[View on GitHub](#)[Download raw](#)

When working with language models, quite often all you need for solid performance is a good prompt and the right tools. However, when dealing with many unique flows, things may get hairy. This cookbook will walk through one way to tackle this.

We'll introduce the notion of **routines** and **handoffs**, then walk through the implementation and show how they can be used to orchestrate multiple agents in a simple, powerful, and controllable way.

Finally, we provide a sample repo, [Swarm](#), that implements these ideas along with examples.

Let's start by setting up our imports.

```
from openai import OpenAI
from pydantic import BaseModel
from typing import Optional
import json
```

```
client = OpenAI()
```



Routines

Ask AI

Docs agent



Loading docs agent...

```
def look_up_item(search_query):  
    """Use to find item ID.  
    Search query can be a description or keywords."""  
  
    # return hard-coded item ID - in reality would be a lookup  
    return "item_132612938"  
  
def execute_refund(item_id, reason="not provided"):  
  
    print("Summary:", item_id, reason) # lazy summary  
    return "success"
```

Docs agent

Loading docs agent...

```
messages = []  
while True:  
    user = input("User: ")  
    messages.append({"role": "user", "content": user})  
  
    run_full_turn(system_message, messages)
```

As you can see, this currently ignores function calls, so let's add that.

Models require functions to be formatted as a function schema. For convenience, we can define a helper function that turns python functions into the corresponding function schema.

Docs agent

Loading docs agent...

```
        f"Unknown type annotation {param.annotation} for parameter\n        {param.name}: {str(e)}"\n    )\n    parameters[param.name] = {"type": param_type}\n\n    required = [\n        param.name\n        for param in signature.parameters.values()\n        if param.default == inspect._empty\n    ]\n\n    return {\n        "type": "function",\n        "function": {
```

Docs agent

Loading docs agent...

```
"name": "sample_function",
"description": "This is my docstring. Call this function when you want.",
"parameters": {
  "type": "object",
  "properties": {
    "param_1": {
      "type": "string"
    },
    "param_2": {
      "type": "string"
    },
    "the_third_one": {
      "type": "integer"
    }
  }
}
```

Docs agent

Loading docs agent...

```
message = response.choices[0].message
```

```
message.tool_calls[0].function
```

```
Function(arguments='{"search_query":"black boot"}', name='look_up_item')
```



Finally, when the model calls a tool we need to execute the corresponding function and provide the result back to the model.

We can do this by mapping the name of the tool to the python function in a `tool_map`, then looking it up in `execute_tool_call` and calling it. Finally we add the result to the conversation.

Docs agent

Loading docs agent...

That response might *also* contain a tool call, so we can just run this in a loop until there are no more tool calls.

If we put everything together, it will look something like this:

```
tools = [execute_refund, look_up_item]
```



```
def run_full_turn(system_message, tools, messages):
```

```
    num_init_messages = len(messages)
```

```
    messages = messages.copy()
```

Docs agent

Loading docs agent...

```
        result_message = {
            "role": "tool",
            "tool_call_id": tool_call.id,
            "content": result,
        }
        messages.append(result_message)

# ==== 3. return new messages =====
return messages[num_init_messages:]

def execute_tool_call(tool_call, tools_map):
    name = tool_call.function.name
```

Docs agent

Loading docs agent...

Handoffs

Let's define a **handoff** as an agent (or routine) handing off an active conversation to another agent, much like when you get transferred to someone else on a phone call. Except in this case, the agents have complete knowledge of your prior conversation!

To see handoffs in action, let's start by defining a basic class for an Agent.

```
class Agent(BaseModel):  
    name: str = "Agent"  
    model: str = "gpt-4o-mini"  
    instructions: str = "You are a helpful Agent"  
    tools: list = []
```



Docs agent

Loading docs agent...

```
if not message.tool_calls: # if finished handling tool calls, break
    break

# === 2. handle tool calls ===

for tool_call in message.tool_calls:
    result = execute_tool_call(tool_call, tools_map)

    result_message = {
        "role": "tool",
        "tool_call_id": tool_call.id,
        "content": result,
    }
```

Docs agent

Loading docs agent...

```
def place_order(item_name):  
    return "success"  
  
sales_assistant = Agent(  
    name="Sales Assistant",  
    instructions="You are a sales assistant. Sell the user a product.",  
    tools=[place_order],  
)  
  
messages = []  
user_query = "Place an order for a black boot."  
print("User:", user_query)  
messages.append({"role": "user", "content": user_query})
```

Docs agent

Loading docs agent...

Handoff Functions

Now that agent can express the *intent* to make a handoff, we must make it actually happen. There's many ways to do this, but there's one particularly clean way.

For the agent functions we've defined so far, like `execute_refund` or `place_order` they return a string, which will be provided to the model. What if instead, we return an `Agent` object to indicate which agent we want to transfer to? Like so:

```
refund_agent = Agent(  
    name="Refund Agent",  
    instructions="You are a refund agent. Help the user with refunds.",  
    tools=[execute_refund],  
)
```



Docs agent

Loading docs agent...

```
messages = messages.copy()

while True:

    # turn python functions into tools and save a reverse map
    tool_schemas = [function_to_schema(tool) for tool in
current_agent.tools]
    tools = {tool.__name__: tool for tool in current_agent.tools}

    # === 1. get openai completion ===
    response = client.chat.completions.create(
        model=agent.model,
        messages=[{"role": "system", "content":
current_agent.instructions}]
```

Docs agent

Loading docs agent...

```
        "tool_call_id": tool_call.id,  
        "content": result,  
    }  
    messages.append(result_message)  
  
    # ==== 3. return last agent used and new messages =====  
    return Response(agent=current_agent,  
                    messages=messages[num_init_messages:])  
  
def execute_tool_call(tool_call, tools, agent_name):  
    name = tool_call.function.name  
    args = json.loads(tool_call.function.arguments)
```

Docs agent

Loading docs agent...

```
def transfer_back_to_triage():  
    """Call this if the user brings up a topic outside of your purview,  
    including escalating to human."""  
    return triage_agent  
  
triage_agent = Agent(  
    name="Triage Agent",  
    instructions=(  
        "You are a customer service bot for ACME Inc. "  
        "Introduce yourself. Always be very brief. "  
        "Gather information to direct the customer to the right department. "  
        "But make your questions subtle and natural."
```

Docs agent

Loading docs agent...

```
        "1. Ask them about any problems in their life related to catching\n        roadrunners.\n"\n        "2. Casually mention one of ACME's crazy made-up products can\n        help.\n"\n        " - Don't mention price.\n"\n        "3. Once the user is bought in, drop a ridiculous price.\n"\n        "4. Only after everything, and if the user says yes, "\n        "tell them a crazy caveat and execute their order.\n"\n        ""\n\n    ),\n    tools=[execute_order, transfer_back_to_triage],\n)
```

Docs agent

Loading docs agent...

```

        """
        "2. Propose a fix (make one up).\n"
        "3. ONLY if not satesfied, offer a refund.\n"
        "4. If accepted, search for the ID and then execute refund."
        """
    ),
    tools=[execute_refund, look_up_item, transfer_back_to_triage],
)

```

Finally, we can run this in a loop (this won't run in python notebooks, so you can try this in a separate python file):

```

agent = triage_agent
messages = []

```



Docs agent

Loading docs agent...